Statistical and Machine Learning

Clustering analysis

Katarzyna Reluga

Clustering analysis

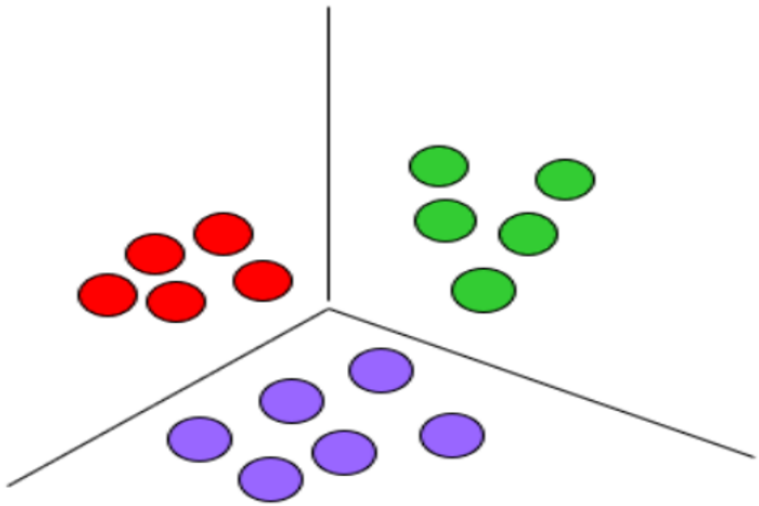
- Introduction
- Evaluating clustering
- Hierarchical: Agglomerative clustering
- Centroid-based: K-means clustering
- Model-based clustering:
  - Mixture model
  - EM-algorithm
- Graph-based: Spectral clustering

Clustering analysis

- Introduction
- Evaluating clustering
- Hierarchical: Agglomerative clustering
- Centroid-based: K-means clustering
- Model-based clustering:
  - Mixture model
  - EM-algorithm
- Graph-based: Spectral clustering

Clustering analysis – introduction

- **Clustering** is a common form of unsupervised learning. The goal is to assign similar data points to the same cluster.
- Intuitively, we want clusters such that:
  - they are compact,
  - the distance between observations **within** a cluster is **small**,
  - the distance between observations **between** different clusters is **large**



Clustering analysis – introduction

- The input is often a dataset of samples: $\mathcal{D} = \{x_n : n = 1 : N\}$, $x_n \in \mathcal{X}$ where typically: $\mathcal{X} = \mathbb{R}^D$
- Equivalently, if we are given data points: $x_1, \ldots, x_n$ or a data matrix: $\mathbf{X}$ clustering aims to find an assignment function: $a(\cdot)$ that maps each observation to one of K clusters: $a(x_i) \in \{1, \ldots, K\}$
- The assignment function should satisfy: $a(x_i) = a(x_j) \iff x_i$ and x_j are similar, i.e., two observations should be assigned to the same cluster when they are similar.
- Another possible input is an $N \times N$ pairwise dissimilarity matrix: $D_{ij} \geq 0$ where D_{ij} measures how dissimilar data points i and j are.
- For example, for qunatitiative variables D_{ij} can be set to be Euclidean, Manhattan, Minkowski, Mahalanobis distance, etc.

Clustering analysis

- Introduction
- Evaluating clustering
- Hierarchical: Agglomerative clustering
- Centroid-based: K-means clustering
- Model-based clustering:
  - Mixture model
  - EM-algorithm
- Graph-based: Spectral clustering

Evaluating clustering: general remarks

- Evaluating clustering quality is difficult because clustering is usually unsupervised.
- If labels are available for some data points, label similarity can be used to assess whether two points should belong to the same cluster.
- If labels are unavailable but the clustering method is based on a generative model, log-likelihood can be used as an evaluation metric.
- Examples of evaluations criteria we discuss in this lecture: purity, Rand index.

Evaluating clustering: the setup

Goal: measure how well a clustering agrees with known class labels.

- Let $U = \{u_1, \ldots, u_R\}$ be the **estimated** clustering (R clusters)
- Let $V = \{v_1, \ldots, v_C\}$ be the **reference** partition derived from true class labels (C classes)
- N = total number of objects; N_i = size of cluster i

Two complementary approaches:

Measure	Perspective
Purity	How homogeneous is each cluster?
Rand Index	How consistent are pairwise assignments?

Purity

Notation: Let N_{ij} = number of objects in cluster i belonging to class j. Define the empirical class distribution within cluster i:

$$p_{ij} = \frac{N_{ij}}{N_i}, \qquad N_i = \sum_{j=1}^{C} N_{ij}$$

The **purity** of cluster i is its dominant class proportion:

$$p_i \triangleq \max_j \; p_{ij}$$

The **overall purity** of a clustering is the weighted average:

$$\text{purity} \triangleq \sum_i \frac{N_i}{N} p_i$$

Intuition: a cluster is pure if nearly all its members share one label. purity $\in [0, 1]$: higher is better.

Purity: Worked Example

Three clusters with $N = 17$ objects and true labels A, B, C:

Cluster	Contents	N_i	Dominant class	p_i
1	A A A A A B	6	A (5/6)	5/6
2	A B B B B C	6	B (4/6)	4/6
3	A A C C C	5	C (3/5)	3/5

$$\text{purity} = \frac{6}{17} \cdot \frac{5}{6} + \frac{6}{17} \cdot \frac{4}{6} + \frac{5}{17} \cdot \frac{3}{5} = \frac{5+4+3}{17} \approx 0.71$$

Limitation: purity is trivially 1 if every object gets its own cluster. It does **not** penalise having too many clusters.

Rand Index: Pairwise Perspective

Idea: consider all $\binom{N}{2}$ pairs of objects and classify each pair into a 2×2 contingency table.

	Same cluster in V	Diff. cluster in V
Same cluster in U	TP	FP
Diff. cluster in U	FN	TN

- TP: pair grouped together in **both** U and V
- TN: pair separated in **both** U and V
- FP: together in U, separated in V
- FN: separated in U, together in V

The **Rand index** is: $R \triangleq \frac{TP+TN}{TP+FP+FN+TN}$

Connection to accuracy: R is exactly the **accuracy** of pairwise clustering decisions read from the confusion matrix above: fraction of all pairs correctly classified as *same* or *different*.

Rand Index: Worked Example

Recall that we need to compare two partitions:

- U: estimated clustering (in our case, pairs in 3 clusters)
- V: true-label partition (in our case, pairs in 3 classes A, B, C)

Total observations: $N = 17$, total pairs: $\binom{17}{2} = 136$, clusters of size 6, 6, 5, then we compute $TP + FP$, i.e., all pairs placed in the same estimated cluster U (see the confusion matrix).

$$TP + FP = \binom{6}{2} + \binom{6}{2} + \binom{5}{2} = 15 + 15 + 10 = 40$$

Rand Index: Worked Example

To get the Rand index, we still need $FN + TN$ (denominator: $TP + FP + FN + TN$)

To obtain FN we can frist compute TP, i.e., the number of pairs in the same estimated cluster and sharing the same true label:

$$TP = \binom{5}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2} = 10 + 6 + 3 + 1 = 20$$

Therefore $FP = TP + FP - TP = 40 - 20 = 20$

Comment:
False positives are pairs together in U, but separated in V.
Example: an A and a B inside the same estimated cluster.

Rand Index: Worked Example

Class totals are: $A = 8$, $B = 5$, $C = 4$. So $TP + FN$, i.e., the number of pairs that share the same true label is

$$TP + FN = \binom{8}{2} + \binom{5}{2} + \binom{4}{2} = 28 + 10 + 6 = 44$$

Then, $FN = TP + FN - TP = 44 - 20 = 24$. False negatives are pairs separated in U, but together in V. Example: an A in cluster 1 and an A in cluster 3.

All pairs fall into one of four categories: $TP + FP + FN + TN = 136$. Therefore $TN = 136 - 20 - 20 - 24 = 72$ (true negatives are pairs separated in both U and V)

Finally, we have

$$R = \frac{20 + 72}{20 + 20 + 24 + 72} = \frac{92}{136} \approx 0.68$$

Clustering analysis

- Introduction
- Evaluating clustering
- Hierarchical: Agglomerative clustering
- Centroid-based: K-means clustering
- Model-based clustering:
  - Mixture model
  - EM-algorithm
- Graph-based: Spectral clustering

Hierarchical Agglomerative Clustering (HAC)

Idea: build a hierarchy of clusters bottom-up by successively merging the most similar groups.

- Input: $N \times N$ dissimilarity matrix $D_{ij} \geq 0$
- Output: a binary tree (**dendrogram**) grouping objects with small dissimilarity hierarchically

Cutting the dendrogram at different heights yields different numbers of (nested) clusters.

Agglomerative clustering – sketch of the algorithm

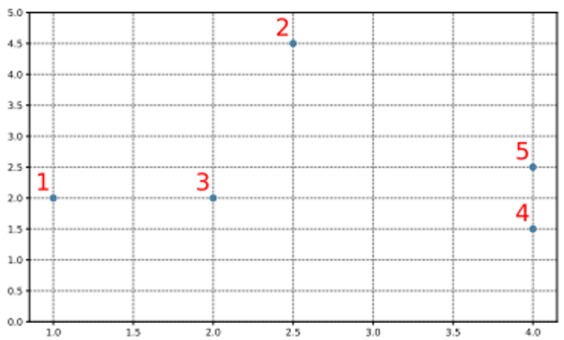
1. Start with n singleton clusters: $C_i = \{i\}, \quad i = 1, \ldots, n$.
2. Maintain a set S of clusters that are still available for merging.
3. Find the pair of clusters with minimum dissimilarity: $(j, k) = \arg\min_{j,k \in S} d_{j,k}$.
4. Merge them into a new cluster: $C_\ell = C_j \cup C_k$.
5. Remove clusters j and k from the available set.
6. If the new cluster does not yet contain all observations, add it back to the available set.
7. Update the dissimilarities between the new cluster C_ℓ and every remaining available cluster.
8. Repeat until the clustering hierarchy is complete.

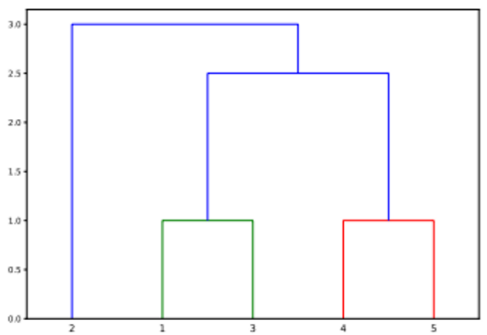
The dendrogram – example

Setup: 5 points $\boldsymbol{x}_n \in \mathbb{R}^2$, dissimilarity = city block distance: $d_{ij} = \sum_{k=1}^{2} |x_{ik} - x_{jk}|$

- Pairs $(1, 3)$ and $(4, 5)$ are both distance 1 apart $\Rightarrow$ merged first
- Sets $\{1, 3\}$, $\{4, 5\}$, $\{2\}$ are then compared and merged iteratively
- Result: the dendrogram below.

Key property: the *height* of each internal node in the dendrogram equals the dissimilarity at which the two branches were merged. Cutting at height h gives all clusters formed before dissimilarity h.

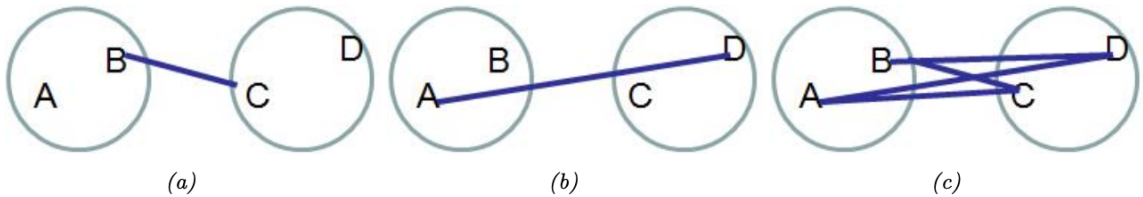




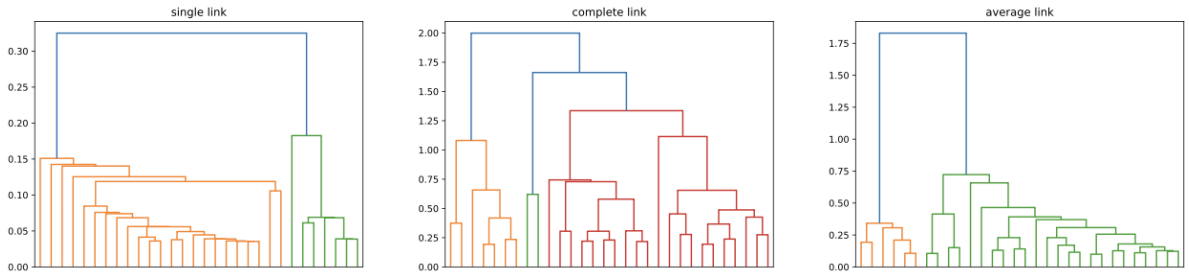
Linkage Criteria: Defining Inter-Cluster Distance

The key choice in HAC is how to measure dissimilarity $d(G, H)$ between two *groups* G and H. Let n_G, n_H be the sizes of groups G and H. We have following types of clustering:

- **Single** link : $d_{SL}(G, H) = \min_{i \in G,\, i' \in H} d_{i,i'}$ (nearest neighbour)
- **Complete** link: $d_{CL}(G, H) = \max_{i \in G,\, i' \in H} d_{i,i'}$ (furthest neighbour)
- **Average** link: $d_{avg}(G, H) = \frac{1}{n_G n_H} \sum_{i \in G} \sum_{i' \in H} d_{i,i'}$



(a) (b) (c)



(a) (b) (c)

Linkage Criteria: Properties and Trade-offs

Single linkage merges groups if *any* pair of members is close.

- Tends to produce elongated, chained clusters (violates **compactness**)
- Recall: diameter of the group $d_G = \max_{i,i' \in G} d_{i,i'}$; single linkage can yield large-diameter clusters

Complete linkage merges groups only if *all* members are close.

- Produces **compact** clusters with small diameter
- Sensitive to outliers (one distant pair can prevent a merge)

Average linkage averages over *all* cross-group pairs.

- Tends to produce clusters that are both compact and well-separated
- **Not invariant** to monotonic rescaling of $d_{i,i'}$ (unlike single and complete linkage)
- Generally the preferred method in practice.

HAC: Pros, Cons and Practical Notes

Advantages

- No need to pre-specify the number of clusters K
- Dendrogram gives a rich, interpretable summary of the data structure
- Flexible: any dissimilarity matrix can be used (not just Euclidean)
- Deterministic: no random initialisation, results are reproducible

Limitations

- Merges are **irreversible**: an early wrong merge cannot be undone
- Scalability: $O(N^2)$–$O(N^3)$ in time and $O(N^2)$ in memory (computationally expensive for large data sets)
- The dendrogram can be sensitive to the choice of linkage, dissimilarity measure, outliers
- No probabilistic model $\Rightarrow$ harder to assess uncertainty or compare models formally

Clustering analysis

- Introduction
- Evaluating clustering
- Hierarchical: Agglomerative clustering
- Centroid-based: K-means clustering
- Model-based clustering:
  - Mixture model
  - EM-algorithm
- Graph-based: Spectral clustering

Motivation: Why Not HAC?

Hierarchical agglomerative clustering has three key limitations:

- **Speed:** $O(N^3)$ time (average linkage) — prohibitive for large datasets
- **Input:** requires a pre-computed dissimilarity matrix; the right notion of similarity is often unclear
- **No objective:** it is purely algorithmic — there is no well-defined cost function being optimised

K-means addresses all three:

- **Speed**: $O(NKT)$ time, T = number of iterations
- **Similarity**: Euclidean distance to learned centroids $\boldsymbol{\mu}_k \in \mathbb{R}^D$
- **Objective**: Minimises a well-defined cost function (distortion)

The K-means Algorithm

Setup: K cluster centres $\boldsymbol{\mu}_k \in \mathbb{R}^D$, assign points to the nearest $\boldsymbol{\mu}_k$:

$$z_n^* = \arg\min_k \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|_2^2$$

Update centres as the mean of all assigned points:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n:\, z_n = k} \boldsymbol{x}_n$$

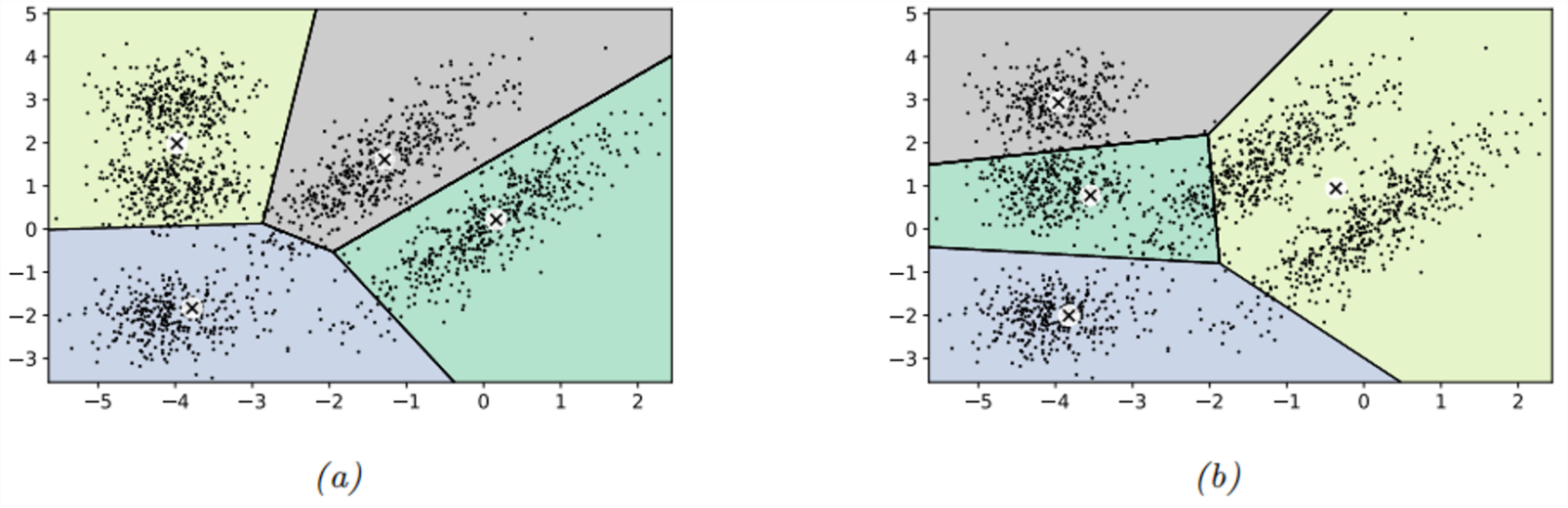
Iterate until convergence. This is **alternating minimisation** of the **distortion**:

$$J(\mathbf{M}, \mathbf{Z}) = \sum_{n=1}^{N} \|\boldsymbol{x}_n - \boldsymbol{\mu}_{z_n}\|^2 = \|\mathbf{X} - \mathbf{Z}\mathbf{M}^\top\|_F^2$$

where $\mathbf{X} \in \mathbb{R}^{N \times D}$, $\mathbf{Z} \in [0,1]^{N \times K}$, $\mathbf{M} \in \mathbb{R}^{D \times K}$ holds the $\boldsymbol{\mu}_k$ as columns.

Note: K-means converges to a *local* minimum of J — initialisation matters.

Illustration of K-means clustering in 2d



- K-means clustering applied to some points in the 2d plane.
- The method induces a Voronoi tessellation of the points.
- The resulting clustering is sensitive to the initialization.
- The lower quality clustering on the right has higher distortion.

Initialisation: K-means++ (farthest point clustering)

K-means optimises a non-convex objective $\Rightarrow$ the result depends on initialisation.

Naive approach: pick K data points randomly as starting centres. Can be improved with **multiple restarts**, but it is slow.

K-means++ : pick centres sequentially to *cover* the data space.

At step t, pick the next centre $\boldsymbol{\mu}_t = \boldsymbol{x}_n$ with probability proportional to its squared distance to the closest existing centre:

$$p(\boldsymbol{\mu}_t = \boldsymbol{x}_n) = \frac{D_{t-1}(\boldsymbol{x}_n)}{\sum_{n'=1}^{N} D_{t-1}(\boldsymbol{x}_{n'})}, \quad D_{t-1}(\boldsymbol{x}) = \min_{k=1}^{t-1} \|\boldsymbol{x} - \boldsymbol{\mu}_k\|_2^2$$

where $D_{t-1}(x)$ is the squared distance of $\boldsymbol{x}$ to the closest existing centroid up to time step $t-1$. Points far from existing centres are more likely to be selected $\Rightarrow$ reduces distortion.

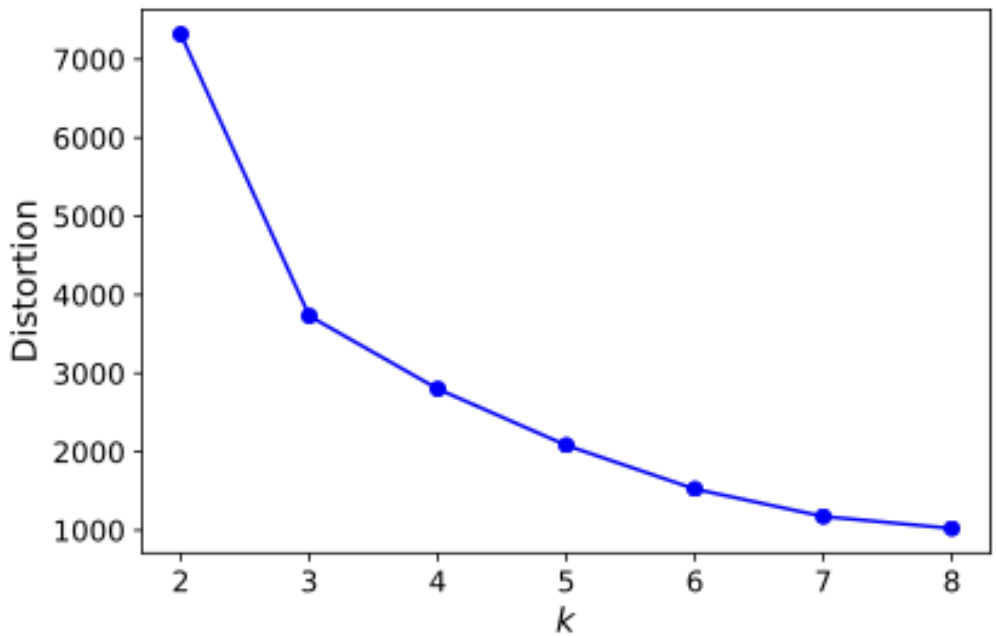
Guarantee: the reconstruction error is at most $O(\log K)$ worse than optimal.

Choosing K: the elbow

Why not minimise distortion? J decreases *monotonically* with K — trivially zero at $K = N$. Validation error fails for the same reason: K-means is a degenerate density model with spikes at $\boldsymbol{\mu}_k$.

Elbow method: plot J vs K; look for a "kink' '. Unreliable in practice — the elbow is often subtle.

Example: Distortion on validation set vs K. There is a slight "kink" or "elbow" in the curve at $K = 3$, but this is hard to detect.



Choosing K: the silhouette

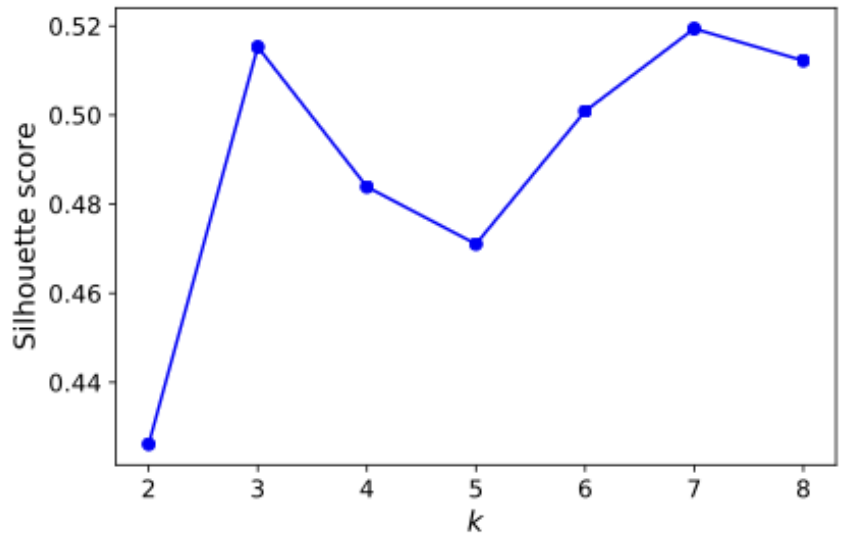
Silhouette coefficient for instance i: $sc(i) = \frac{b_i - a_i}{\max(a_i, b_i)}$

where a_i = mean distance to other members of i's cluster (compactness), b_i = mean distance to members of the *next closest* cluster (separation).

$sc(i) \approx +1$: well-clustered; $sc(i) \approx 0$: near boundary; $sc(i) \approx -1$: likely misassigned

The **silhouette score** of a clustering is the mean $sc(i)$ over all instances. Choose K that maximises it.

Example: Silhouette score vs K: peaks at $K = 3$ and $K = 7$.



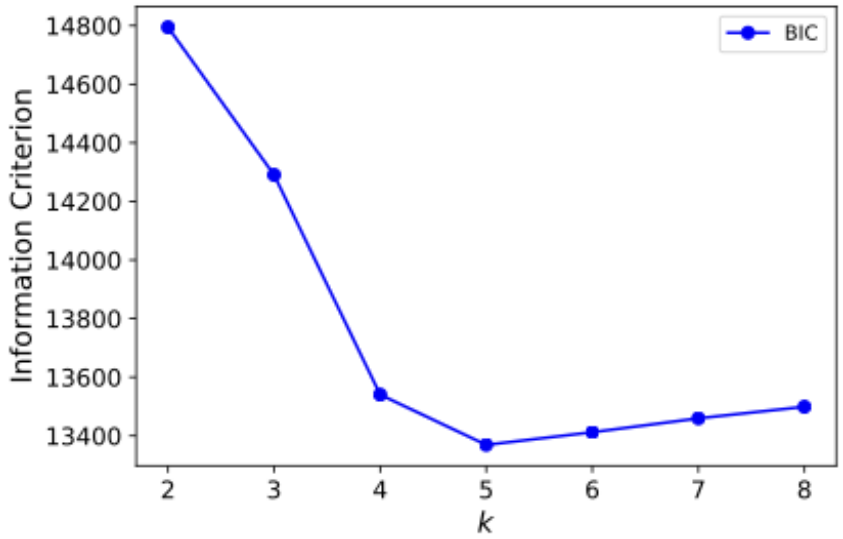
Choosing K: marginal likelihood

Marginal likelihood (BIC): fit a proper probabilistic model (e.g. a Guassian mixture model, see following slides) and use:

$$\mathrm{BIC}(K) = \log p(\mathcal{D} \mid \hat{\boldsymbol{\theta}}_K) - \frac{D_K}{2} \log(N)$$

where $p(\mathcal{D} \mid \hat{\boldsymbol{\theta}}_K)$ is the likelihood of the dataset under the fitted model., D_K = number of parameters, $\hat{\boldsymbol{\theta}}_K$ = MLE.

BIC exhibits a U-shaped curve — once enough clusters cover the true modes, Occam's razor penalises further complexity (see the example below)



Summary: K-means pros and cons

Advantages

- Simple and fast: $O(NKT)$
- Well-defined objective J
- Scales to large datasets
- K-means++ gives theoretical guarantees

Limitations

- K must be specified in advance
- Converges to local minima only (depends on starting values)
- Assumes spherical, equally-sized clusters
- Sensitive to outliers and scale

Clustering

- Introduction
- Hierarchical: Agglomerative clustering
- Centroid-based: K-means clustering
- Model-based clustering:
  - Mixture model
  - EM-algorithm
- Graph-based: Spectral clustering

Mixture Models

Idea: build complex probability distributions as convex combinations of simpler ones.

A **mixture model** has the form:

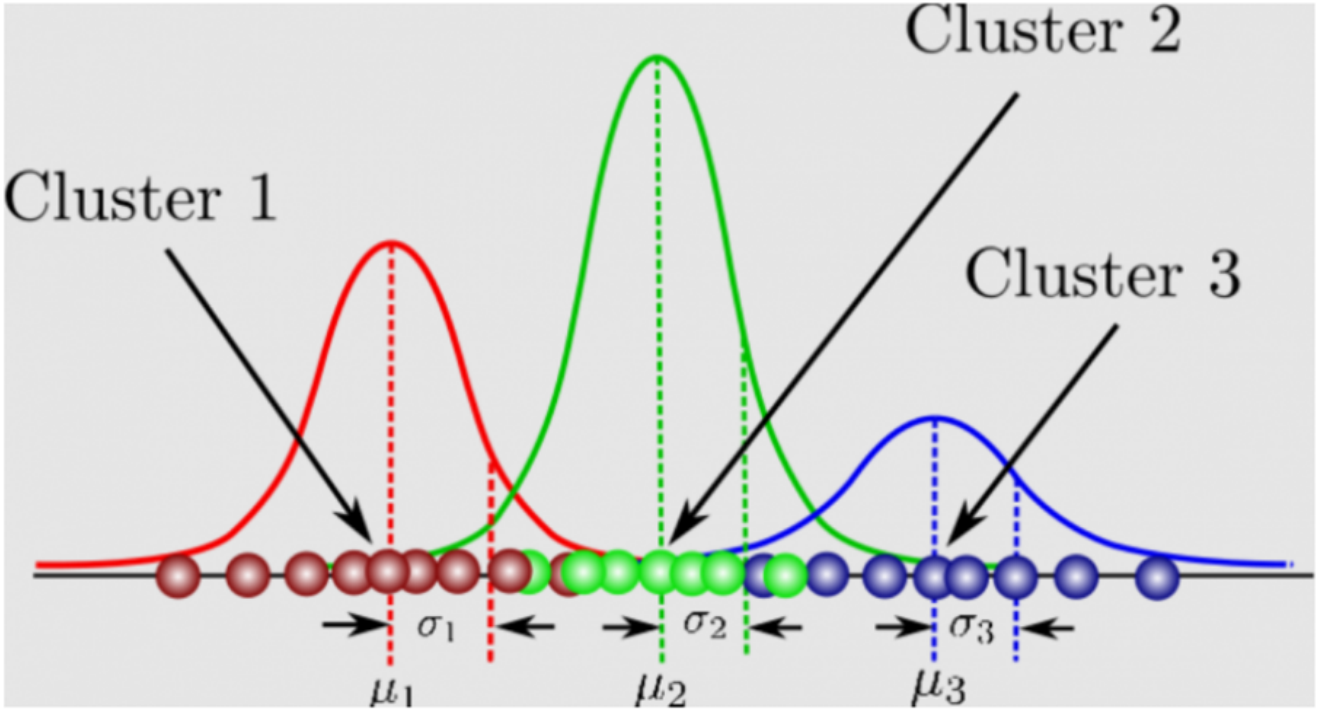
$$p(\boldsymbol{y} \mid \boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k p_k(\boldsymbol{y})$$

- $p_k(\boldsymbol{y})$ is the k-th **mixture component**
- π_k are the **mixture weights**, satisfying $0 \leq \pi_k \leq 1$ and $\sum_{k=1}^{K} \pi_k = 1$

Intuition: the observed data is assumed to arise from one of K subpopulations, with probability π_k of belonging to subpopulation k. The identity of the subpopulation is a **latent (hidden) variable** $z_n \in \{1, \ldots, K\}$.

With sufficiently many components, a mixture model can approximate any smooth distribution over $\mathbb{R}^D$.

Mixture Models – illustration



Gaussian Mixture Model (GMM)

A **Gaussian mixture model (GMM)**, also called a **mixture of Gaussians (MoG)**, uses multivariate Gaussians as components:

$$p(\boldsymbol{x} \mid \boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Parameters: $\boldsymbol{\theta} = (\boldsymbol{\pi}, \{\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\})$

GMMs for clustering: fit the model by computing the MLE $\hat{\boldsymbol{\theta}} = \arg\max \log p(\mathcal{D} \mid \boldsymbol{\theta})$, then assign each point $\boldsymbol{x}_n$ to the most probable component.

Responsibilities: posterior cluster membership

Given parameters $\boldsymbol{\theta}$, we can compute the **responsibility** of cluster k for data point $\boldsymbol{x}_n$ via Bayes' rule:

$$r_{nk} \triangleq p(z_n = k \mid \boldsymbol{x}_n, \boldsymbol{\theta}) = \frac{p(z_n = k \mid \boldsymbol{\theta})\, p(\boldsymbol{x}_n \mid z_n = k, \boldsymbol{\theta})}{\sum_{k'=1}^{K} p(z_n = k' \mid \boldsymbol{\theta})\, p(\boldsymbol{x}_n \mid z_n = k', \boldsymbol{\theta})}$$

r_{nk} is the posterior probability that point n was generated by component k.

Hard vs soft cluster assignemnt:

Type	Assignment rule
Soft	Use responsibilities r_{nk} fractionally
Hard	Assign $\hat{z}_n = \arg\max_k \; r_{nk}$

Responsibilities: posterior cluster membership

The most probable (**hard**) cluster assignment is:

$$\hat{z}_n = \arg\max_k r_{nk} = \arg\max_k \left[\log p(\boldsymbol{x}_n \mid z_n = k, \boldsymbol{\theta}) + \log p(z_n = k \mid \boldsymbol{\theta})\right]$$

Special case: with uniform prior $\pi_k = 1/K$ and spherical covariances $\boldsymbol{\Sigma}_k = \mathbf{I}$, hard clustering reduces to K-means.

Problem

To compute r_{nk} we need $\boldsymbol{\theta}$; to estimate $\boldsymbol{\theta}$ we need r_{nk}. This circular dependency calls for an iterative algorithm $\rightarrow$ the **EM algorithm**.

The EM algorithm: intuition

- An iterative algorithm to consist two parts:
- Expectation (E-step): calculates posterior probability to estimate the probability that a point belongs to a cluster $\rightarrow$ assigns data to the cluster.
- Maximization (M-step): estimates the parameters for each cluster.
- EM algorithm iterates between these two steps until convergence.
- EM algorithm can be viewed as a "soft" version of k-means

The EM algorithm: motivation

Goal: maximise the observed-data log likelihood:

$$\ell(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log p(\boldsymbol{y}_n \mid \boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{\boldsymbol{z}_n} p(\boldsymbol{y}_n, \boldsymbol{z}_n \mid \boldsymbol{\theta}) \right]$$

Difficulty: the log of a sum cannot be simplified — direct optimisation is intractable.

Key idea (MM / bound optimisation): instead of maximising $\ell(\boldsymbol{\theta})$ directly, construct a **surrogate function** $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t)$ that:

- Is a tight lower bound: $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) \leq \ell(\boldsymbol{\theta})$ for all $\boldsymbol{\theta}$
- Is tight at current estimate: $Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \ell(\boldsymbol{\theta}^t)$

Then update: $\boldsymbol{\theta}^{t+1} = \arg\max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t)$

This guarantees **monotonic increase**: $\ell(\boldsymbol{\theta}^{t+1}) \geq \ell(\boldsymbol{\theta}^t)$.

The EM algorithm: the ELBO

Introduce auxiliary distributions $q_n(\boldsymbol{z}_n)$ over each hidden variable. By **Jensen's inequality** (log is concave):

$$\ell(\boldsymbol{\theta}) \geq \sum_n \sum_{\boldsymbol{z}_n} q_n(\boldsymbol{z}_n) \log \frac{p(\boldsymbol{y}_n, \boldsymbol{z}_n \mid \boldsymbol{\theta})}{q_n(\boldsymbol{z}_n)}$$

$$= \sum_n \underbrace{\mathbb{E}_{q_n}\left[\log p(\boldsymbol{y}_n, \boldsymbol{z}_n \mid \boldsymbol{\theta})\right] + \mathbb{H}(q_n)}_{Ł(\boldsymbol{\theta}, q_n)} = Ł(\boldsymbol{\theta}, q_{1:N})$$

This lower bound is called the **evidence lower bound (ELBO)**.

The bound is **tight** when $q_n(\boldsymbol{z}_n) = p(\boldsymbol{z}_n \mid \boldsymbol{y}_n, \boldsymbol{\theta})$, since the KL divergence $D_{\mathrm{KL}}(q_n \| p(\boldsymbol{z}_n \mid \boldsymbol{y}_n, \boldsymbol{\theta})) = 0$ iff $q_n = p(\boldsymbol{z}_n \mid \boldsymbol{y}_n, \boldsymbol{\theta})$.

The EM Algorithm: E and M Steps

The EM algorithm alternates between two steps:

E step (Expectation): fix $\boldsymbol{\theta}^t$ and tighten the bound by setting

$$q_n^*(\boldsymbol{z}_n) = p(\boldsymbol{z}_n \mid \boldsymbol{y}_n, \boldsymbol{\theta}^t)$$

This sets $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) = Ł(\boldsymbol{\theta}, \{q_n^*\})$, making the ELBO equal to $\ell(\boldsymbol{\theta}^t)$.

M step (Maximization): fix $\{q_n^t\}$ and maximise the ELBO over $\boldsymbol{\theta}$:

$$\boldsymbol{\theta}^{t+1} = \arg\max_{\boldsymbol{\theta}} \sum_n \mathbb{E}_{q_n^t} \left[\log p(\boldsymbol{y}_n, \boldsymbol{z}_n \mid \boldsymbol{\theta})\right]$$

The entropy terms $\mathbb{H}(q_n)$ are constant in $\boldsymbol{\theta}$ and can be dropped.

Convergence: EM converges to a *local* maximum of $\ell(\boldsymbol{\theta})$.
Monotonic increase is guaranteed at every iteration.

EM for GMM: E Step

Recall the GMM: $p(\boldsymbol{x} \mid \boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

E step: compute the responsibility of cluster k for point n using the current parameters $\boldsymbol{\theta}^{(t)}$:

$$r_{nk}^{(t)} = p^*(z_n = k \mid \boldsymbol{y}_n, \boldsymbol{\theta}^{(t)}) = \frac{\pi_k^{(t)} \, p(\boldsymbol{y}_n \mid \boldsymbol{\theta}_k^{(t)})}{\sum_{k'} \pi_{k'}^{(t)} \, p(\boldsymbol{y}_n \mid \boldsymbol{\theta}_{k'}^{(t)})} \tag{8.160}$$

Each $r_{nk}^{(t)} \in [0, 1]$ and $\sum_k r_{nk}^{(t)} = 1$: these are *soft* memberships.

Define the **effective cluster size**: $r_k^{(t)} \triangleq \sum_n r_{nk}^{(t)}$, the total responsibility assigned to cluster k.

EM for GMM: M Step

M step: maximise the expected complete-data log likelihood over $\boldsymbol{\theta}$, using the responsibilities from the E step. The updates in closed form are:

Means (responsibility-weighted centroid):

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_n r_{nk}^{(t)} \boldsymbol{y}_n}{r_k^{(t)}}$$

Covariances (responsibility-weighted scatter):

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{\sum_n r_{nk}^{(t)} (\boldsymbol{y}_n - \boldsymbol{\mu}_k^{(t+1)})(\boldsymbol{y}_n - \boldsymbol{\mu}_k^{(t+1)})^\top}{r_k^{(t)}}$$

Mixing weights: $\pi_k^{(t+1)} = \frac{1}{N} \sum_n r_{nk}^{(t)} = \frac{r_k^{(t)}}{N}$

These are exactly the MLEs of a Gaussian, but **weighted** by responsibilities rather than hard counts.

GMM vs K-means: EM as Generalisation

K-means is a **special case** of EM:

1. Fix $\boldsymbol{\Sigma}_k = \mathbf{I}$, $\pi_k = 1/K$ for all k (spherical, equal-weight clusters)
2. **Hard E step**: replace soft responsibilities with hard assignments $r_{nk} \approx \mathbb{I}(k = z_n^*)$, $z_n^* = \arg\max_k \; r_{nk}$

With these two approximations, the M step reduces exactly to the K-means centroid update.

Feature	K-means	GMM + EM
Assignment	Hard	Soft
Cluster shape	Spherical	Arbitrary $\boldsymbol{\Sigma}_k$
Objective	Distortion J	Log likelihood $\ell(\boldsymbol{\theta})$
Uncertainty	None	r_{nk} quantifies it
Convergence	Local min of J	Local max of ℓ

EM for GMM: Example (Old Faithful)

Dataset: Old Faithful geyser (Yellowstone) — eruption duration vs. waiting time to next eruption (N standardised 2d observations).

Model: $K = 2$ component GMM, initialised with:

$$\boldsymbol{\mu}_1 = (-1, 1),\ \boldsymbol{\Sigma}_1 = \mathbf{I}, \quad \boldsymbol{\mu}_2 = (1, -1),\ \boldsymbol{\Sigma}_2 = \mathbf{I}$$

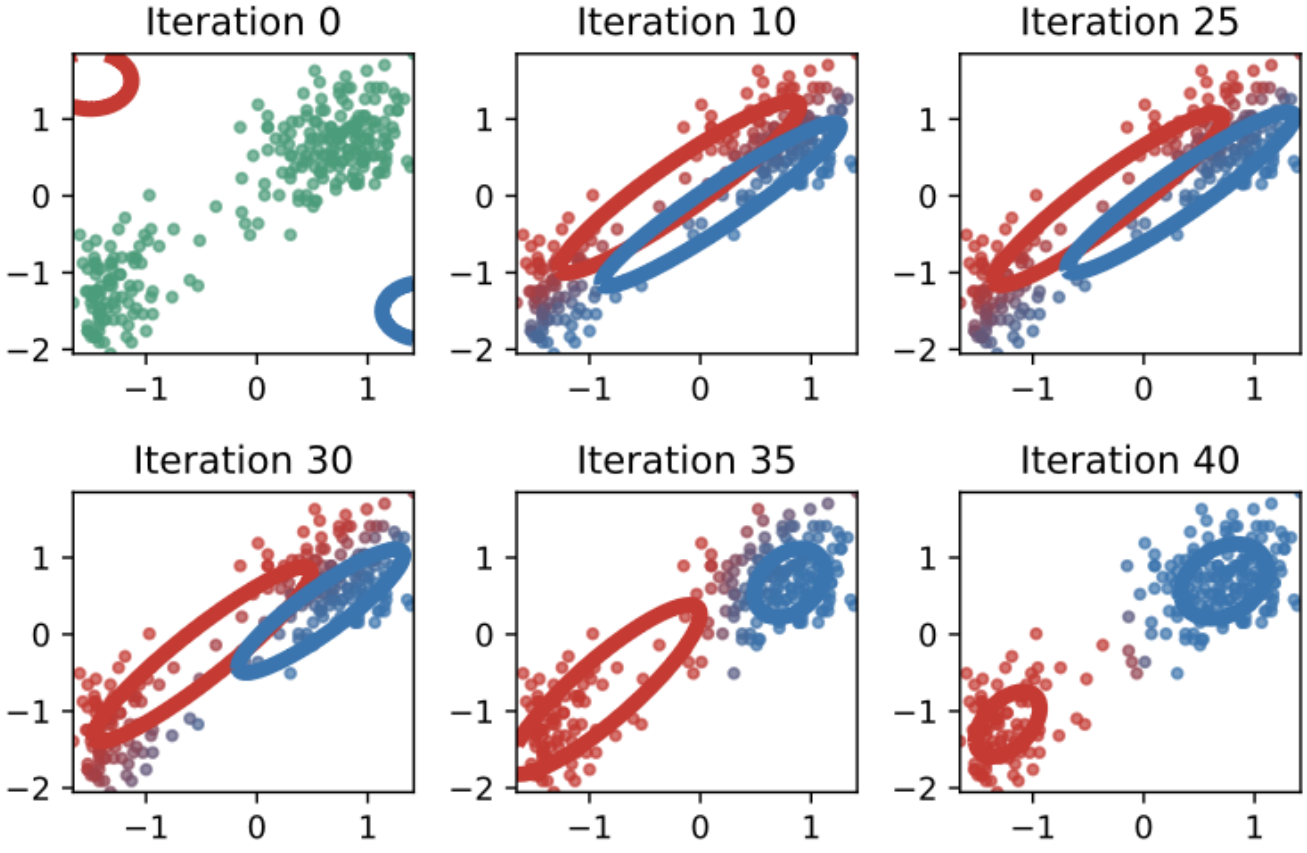
Procedure:

1. **E step:** compute $r_{nk}^{(t)}$ for each point using current $\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \pi_k$
2. **M step:** update $\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \pi_k$ using responsibility-weighted MLEs
3. Repeat until $\ell(\boldsymbol{\theta})$ converges

Result: EM recovers the two well-separated clusters in the data, with the elliptical contours of each Gaussian adapting to the local shape of the data — something K-means (restricted to spherical clusters) cannot do.

EM for GMM: illustration with Old Faithful data

The degree of redness indicates the degree to which the point belongs to the red cluster, and similarly for blue; thus purple points have a roughly 50/50 split in their responsibilities to the two clusters.



Clustering

- Introduction
- Hierarchical: Agglomerative clustering
- Centroid-based: K-means clustering
- Model-based clustering:
  - Mixture model
  - EM-algorithm
- Graph-based: Spectral clustering

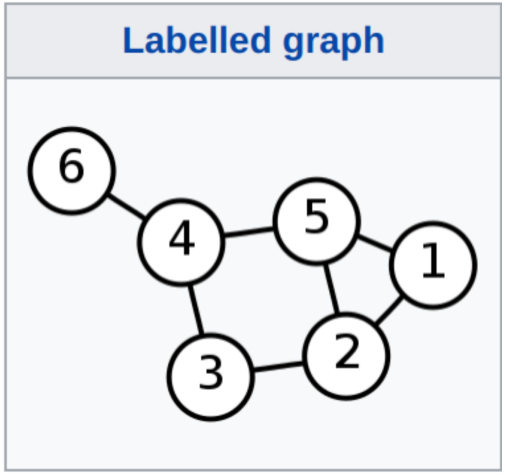
Graphs – basic notions

A **graph** represents relationships between objects.

- Nodes: objects/data points $i = 1, \ldots, n$
- Edges: connections between pairs of nodes

For graph-based learning, each observation $\mathbf{x}_i$ is treated as a node.

An edge between nodes i and j means that $\mathbf{x}_i$ and $\mathbf{x}_j$ are considered similar.



Weight matrix

In graph-based learning, edges often have **weights** measuring similarity.

The **weight matrix** is $W = (w_{ij}) \in \mathbb{R}^{n \times n}$, where $w_{ij} \geq 0$ measures the similarity between nodes i and j.

For an undirected weighted graph,

$$w_{ij} = w_{ji}.$$

A larger w_{ij} means that nodes i and j are more similar.

Examples of graph weights

A common similarity weight is the Gaussian weight:

$$w_{ij} = \exp\left\{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\sigma^2}\right\}.$$

- Nearby points receive large weights.
- Distant points receive small weights.

Alternatively, one can construct a k-nearest-neighbour graph:

$$w_{ij} = 1 \quad \text{if } \mathbf{x}_i \text{ is among the } k \text{ nearest neighbours of } \mathbf{x}_j,$$

or vice versa.

This makes the graph sparse.

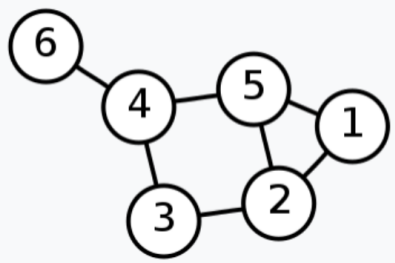
Degree matrix

The **degree** of node i is the total weight of all edges connected to node i: $d_i = \sum_{j=1}^{n} w_{ij}$.

The **degree matrix** is the diagonal matrix

$$D = \text{diag}(d_1, \ldots, d_n) \rightarrow D_{ij} = \begin{cases} d_i, & i = j, \\ 0, & i \neq j. \end{cases}$$

The degree matrix records how strongly each node is connected to the rest of the graph.

Labelled graph	Degree matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$

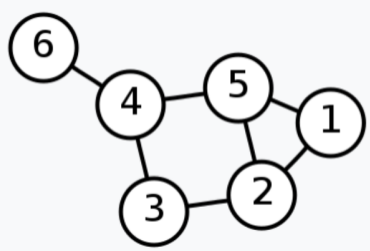
Adjacency matrix

For an unweighted graph, the graph can be represented by an **adjacency matrix** $A = (a_{ij}) \in \mathbb{R}^{n \times n}$.

The entries are

$$a_{ij} = \begin{cases} 1, & \text{if there is an edge between nodes } i \text{ and } j, \\ 0, & \text{otherwise.} \end{cases}$$

For an undirected graph, $a_{ij} = a_{ji}$. Usually, $a_{ii} = 0$, because we do not connect a node to itself.

Labelled graph	Degree matrix	Adjacency matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$

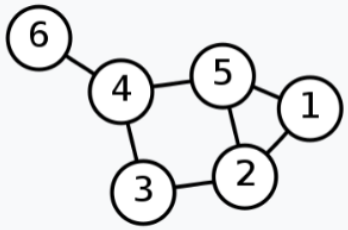
Graph Laplacian

The **graph Laplacian** is defined as $L = D - W$. Its elements are

$$L_{ij} = \begin{cases} d_i, & i = j, \\ -w_{ij}, & i \neq j \text{ and } w_{ij} \neq 0, \\ 0, & \text{otherwise.} \end{cases}$$

The graph Laplacian compares each node with its neighbours.

It is a discrete analogue of a derivative operator on a graph.

Labelled graph	Degree matrix	Adjacency matrix	Laplacian matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$

Spectral Clustering: Motivation

Problem with K-means and GMMs: both assume clusters are convex and roughly spherical. They fail on non-convex, elongated, or ring-shaped data.

Spectral clustering takes a different approach: exploit the geometry of a **pairwise similarity graph**.

Key idea:

- Represent the data as a weighted graph $\mathbf{W}$, where edge weight w_{ij} encodes similarity between points i and j
- Perform eigenvalue analysis of the graph's **Laplacian matrix L**
- Use the resulting eigenvectors as new feature vectors for each point
- Apply K-means in this new feature space

The eigenvectors capture the global connectivity structure of the graph — something Euclidean distance alone cannot.

Graph Construction and the Cut Objective

Step 1: build the similarity graph. Each data point is a node; edge weight $w_{ij} \geq 0$ measures similarity. Typically connect each node only to its most similar neighbours (sparse graph).

Goal: partition nodes into K disjoint sets $S_1, \ldots, S_K$ minimising between-cluster connections.

A natural cost is the **graph cut**:

$$\mathrm{cut}(S_1, \ldots, S_K) \triangleq \frac{1}{2} \sum_{k=1}^{K} W(S_k, \bar{S}_k)$$

$W(A, B) \triangleq \sum_{i \in A, j \in B} w_{ij}$, $\bar{S}_k = V \setminus S_k$ is the complement of S_k.

Problem: minimising the raw cut tends to isolate single nodes. Fix: normalise by cluster size.

The Normalised Cut (Ncut)

Step 2: normalise the cut. Divide the cut weight by the **volume** of each set, $\mathrm{vol}(A) \triangleq \sum_{i \in A} d_i$, where $d_i = \sum_j w_{ij}$ is the weighted degree of node i:

$$\mathrm{Ncut}(S_1, \ldots, S_K) \triangleq \frac{1}{2} \sum_{k=1}^{K} \frac{\mathrm{cut}(S_k, \bar{S}_k)}{\mathrm{vol}(S_k)}$$

Ncut encourages clusters that are:

- **Internally dense** (strong within-cluster connections)
- **Externally sparse** (weak between-cluster connections)
- **Balanced** in total edge weight

Bad news: minimising Ncut exactly requires finding binary indicator vectors $\boldsymbol{c}_i \in \{0, 1\}^N$ — this is **computationally intensive**.

Good news: a continuous relaxation via eigenvectors of the graph Laplacian is tractable and works well in practice.

Properties of the Graph Laplacian

Consider the **graph Laplacian** $\mathbf{L} \triangleq \mathbf{D} - \mathbf{W}$ defined above, where $\mathbf{W}$ is the symmetric similarity matrix and $\mathbf{D} = \mathrm{diag}(d_i)$ is the diagonal degree matrix.

Theorem on Graph Laplacian *The eigenspace of* $\mathbf{L}$ *with eigenvalue* 0 *is spanned by the indicator vectors* $\mathbf{1}_{S_1}, \ldots, \mathbf{1}_{S_K}$*, where* $S_1, \ldots, S_K$ *are the* K *connected components of the graph.*

Intuition: if the graph has K perfectly separated components, $\mathbf{L}$ is block diagonal, and its K zero-eigenvectors are exactly the cluster membership indicators. In the noisy case, the K smallest eigenvectors remain approximately piecewise constant — perturbation theory guarantees they stay close to the ideal indicators.

Practical issue: nodes with high degree dominate. Use the **normalised Laplacian** instead.

The normalised Laplacian and algorithm

To correct for degree imbalance, use the **symmetric normalised Laplacian**:

$$\mathbf{L}_{sym} \triangleq \mathbf{D}^{-\frac{1}{2}}\mathbf{L}\mathbf{D}^{-\frac{1}{2}} = \mathbf{I} - \mathbf{D}^{-\frac{1}{2}}\mathbf{W}\mathbf{D}^{-\frac{1}{2}}$$

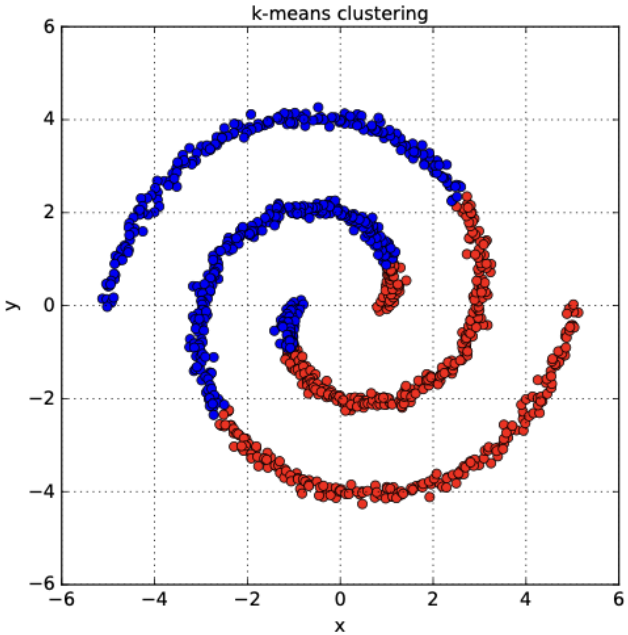
Its zero-eigenspace is spanned by $\mathbf{D}^{\frac{1}{2}}\mathbf{1}_{S_k}$.

Spectral clustering algorithm (Ng–Jordan–Weiss):

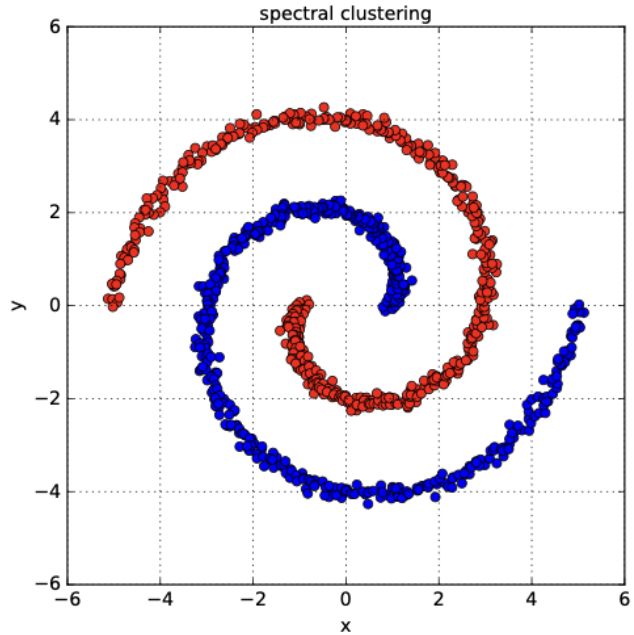
- Build the similarity matrix $\mathbf{W}$ (e.g. Gaussian kernel $w_{ij} = \exp(-\frac{1}{2\sigma^2}\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2)$)
- Compute $\mathbf{L}_{sym}$ and find its K eigenvectors with smallest eigenvalues; stack into $\mathbf{U} \in \mathbb{R}^{N\times K}$
- Row-normalise $\mathbf{U}$: set $t_{ij} = u_{ij}/\sqrt{\sum_k u_{ik}^2}$, forming matrix $\mathbf{T}$
- Apply K-means to the rows $\boldsymbol{t}_i \in \mathbb{R}^K$ of $\mathbf{T}$
- Map cluster assignments back to the original data points

Spectral Clustering: Example

Data with two interleaved rings. K-means (spherical assumption, left) fails completely. Spectral clustering (right) with Gaussian kernel weights $W_{ij} = \exp(-\frac{1}{2\sigma^2}\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2^2)$ and $K = 2$ correctly separates the two rings by working in the eigenvector space of $\mathbf{L}_{sym}$.



(a)



(b)

Spectral Clustering: Pros/Cons

Advantages

- Handles non-convex, non-spherical clusters
- No parametric model assumed
- Works on any similarity matrix, not just Euclidean distances
- Principled connection to graph cuts (Ncut)

Limitations

- K must be specified in advance
- Sensitive to the choice of similarity function and σ
- Eigendecomposition costs $O(N^3)$; memory $O(N^2)$
- Final step still relies on K-means (local minima)

References

- ESLII: Chapter 14.3
- PML: Chapters 3.5, 8.7, 21